

Arquitetura de Software

CST em Análise e Desenvolvimento de Sistemas

Professor: Arliones Hoeller Jr.

arliones.hoeller@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Arquitetura de Software

Projeto Arquitetural

- O projeto é uma atividade criativa.
- Cada arquiteto tem sua própria maneira de projetar o software.
- O projeto arquitetural faz a ligação entre:
 - Requisitos (domínio do problema)
 - Projeto detalhado (domínio da solução)

Definindo a Solução

- Primeiro passo do projeto de sistema
- Frequentemente conduzido em paralelo à especificação de requisitos
- Envolve:
 - Identificação dos componentes principais do sistema
 - Definição das interfaces de comunicação entre os componentes

Vantagens da Arquitetura

- Comunicação entre stakeholders
- Análise de sistema (requisitos não funcionais)
- Reutilização em larga escala

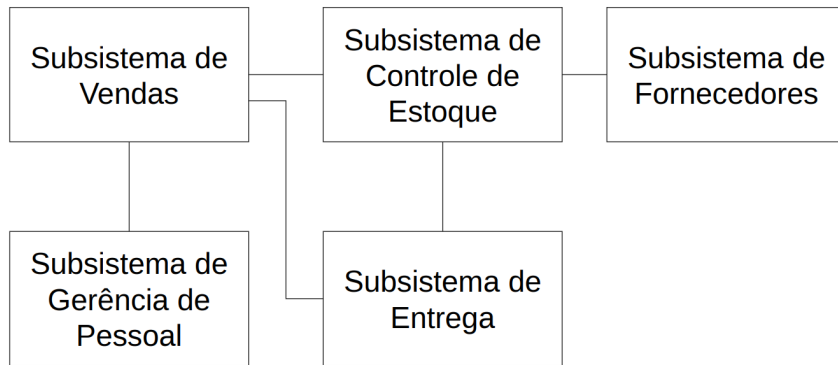
Impacto da Arquitetura

- **Desempenho:** gargalos de comunicação
- **Facilidade de distribuição:** execução em múltiplas máquinas
- **Manutenção:** componentes substituíveis

Diagrama de Componentes

- Identifica os principais subsistemas a serem desenvolvidos
- Notação simplificada:
 - Caixas: subsistemas (componentes)
 - Linhas: comunicação entre subsistemas
 - Caixas dentro de caixas: decomposição

Exemplo de Arquitetura



Quando usar o diagrama?

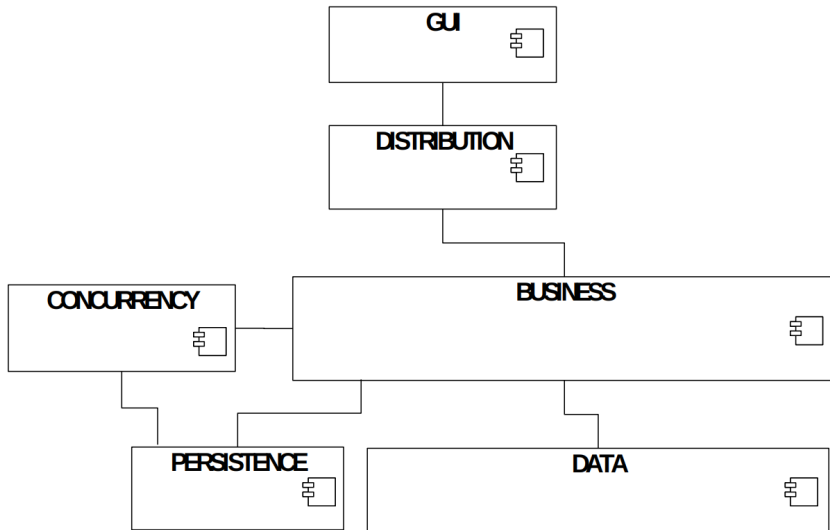
Pontos positivos

- Facilita comunicação com stakeholders
- Útil para planejamento de projeto
- Facilita desenvolvimento em paralelo

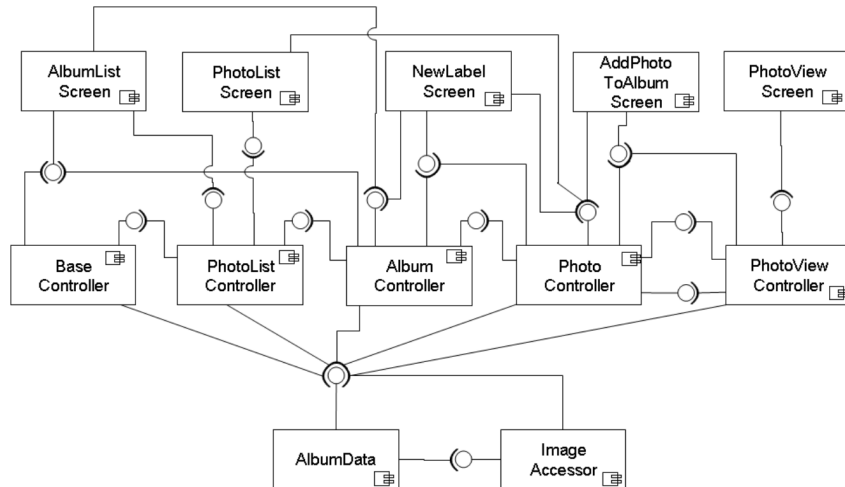
Pontos negativos

- Não mostra a natureza dos relacionamentos
- Não indica propriedades internas dos subsistemas

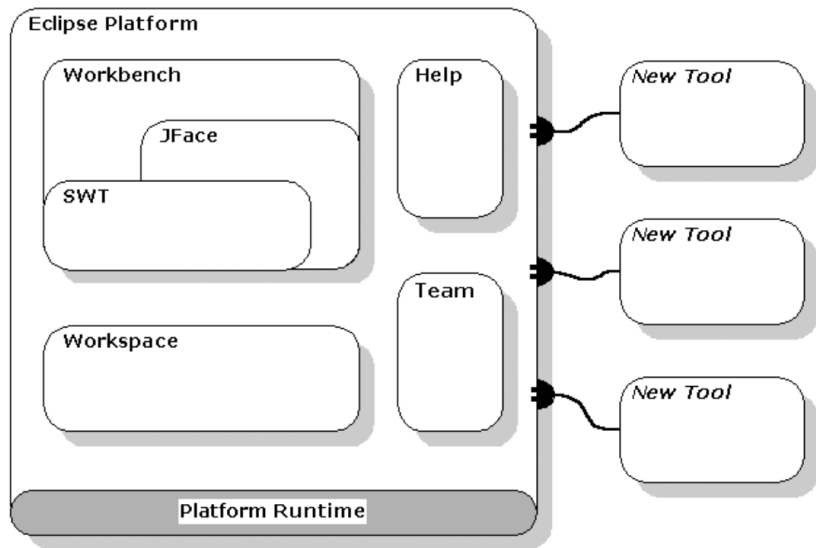
Exemplo de Arquitetura (2)



Exemplo de Arquitetura (3)



Exemplo de Arquitetura (4)



Decisões de Projeto Arquitetural

- Como o sistema será distribuído?
- Como decompor as funcionalidades?
- Como verificar se a arquitetura atende aos requisitos não funcionais?
- Existe uma arquitetura genérica aplicável?

Requisitos Não Funcionais

A arquitetura deve considerar:

- **Desempenho:** evitar comunicação excessiva entre componentes
- **Segurança:** ocultar características críticas
- **Disponibilidade:** tolerância a falhas, redundância

Exercício 1 — Sistema de Agendamento de Consultas Médicas

I

Contexto: Uma clínica médica deseja desenvolver um sistema online que permita aos pacientes agendarem consultas com médicos de diferentes especialidades. O sistema deve permitir:

- Cadastro de pacientes e médicos.
- Visualização de horários disponíveis.
- Agendamento, cancelamento e confirmação de consultas.
- Acesso restrito para médicos verificarem sua agenda.

Exercício 1 — Sistema de Agendamento de Consultas Médicas II

Tarefa:

- Modele um diagrama de componentes representando os principais módulos do sistema.
- Seu diagrama deve conter ao menos: Interface Web ou Aplicativo, Módulo de autenticação, Módulo de agendamento, Módulo de gerenciamento de usuários (pacientes e médicos), Banco de dados.

Dica: Considere separar a camada de interface da lógica de negócios e da persistência de dados. Represente as dependências e conexões entre os componentes.

Exercício 2 — Aplicativo de Entregas I

Contexto: Você foi contratado para projetar um sistema backend para um aplicativo de entregas que será usado por restaurantes, entregadores e clientes. O sistema deve gerenciar:

- Cadastro de restaurantes e seus menus
- Cadastro e pedidos dos clientes
- Atribuição e rastreamento de entregas
- Notificações em tempo real

Exercício 2 — Aplicativo de Entregas II

Tarefa:

- 1 Modele uma arquitetura de componentes adequada para esse sistema.
- 2 Seu diagrama deve conter ao menos: Interfaces externas (aplicativos ou APIs), Serviços principais (ex: pedidos, rastreamento, notificações), Camada de persistência.
- 3 Indique no diagrama como os componentes se comunicam (setas, conexões etc.).
- 4 Especifique brevemente (em texto) como sua arquitetura considera escalabilidade e resiliência.

Padrões de Arquitetura de Software

O que é um Padrão?

- Uma descrição de um problema e a essência de sua solução.
- Documenta boas soluções para problemas recorrentes.
- Deve ser suficientemente abstrato para ser reutilizado em diferentes aplicações.
- Arquiteturas de Referência
 - Cada sistema é único, mas sistemas de um mesmo domínio frequentemente têm arquiteturas semelhantes.
 - Um projeto pode se beneficiar de um padrão arquitetural conhecido.

O que são Padrões Arquiteturais?

- Formas de organizar a estrutura fundamental de um sistema.
- Permitem construir arquiteturas aderentes a certas propriedades.
- Auxiliam na definição da arquitetura do sistema.
- Elementos de um Padrão Arquitetural
 - Sub-sistemas e suas responsabilidades.
 - Regras de relacionamento entre os sub-sistemas.
 - A escolha do padrão pode ser o primeiro passo na solução arquitetural.

Da Arquitetura à Implementação

- Padrões arquiteturais definem a estrutura mais abstrata do sistema.
- Fases subsequentes devem seguir essa estrutura:
 - Padrões de projeto
 - Linguagens de programação
- Refinamento de Padrões
 - Padrões são templates e não definem completamente a arquitetura.
 - Devem ser refinados:
 - Inclusão de componentes adicionais.
 - Definição de padrões de projeto e de programação.

Escolhendo um Padrão

A escolha deve considerar:

- Tipo de sistema.
- Requisitos não funcionais (confiabilidade, adaptabilidade etc.).
- Questões como:
 - É um sistema interativo?
 - Possui muitas variações?
- Composição de Padrões
 - Sistemas complexos podem combinar diferentes padrões.
 - Diferentes padrões levam a consequências distintas, mesmo para problemas semelhantes.

Padrões Arquiteturais: Exemplos

■ Da desordem à estrutura:

- Arquitetura em Camadas (Layered Architecture)
- Arquitetura de Repositório (Blackboard)
- Dutos e Filtros (Pipes and Filters)

■ Sistemas Distribuídos:

- Cliente-Servidor (Client-Server)
- Broker

■ Sistemas Interativos:

- Model-View-Controller (MVC)
- Presentation-Abstraction-Control (PAC)

■ Sistemas Adaptáveis:

- Event-Driven Architecture (EDA)
- Microkernel
- Reflection

Exemplos de Padrões Arquiteturais

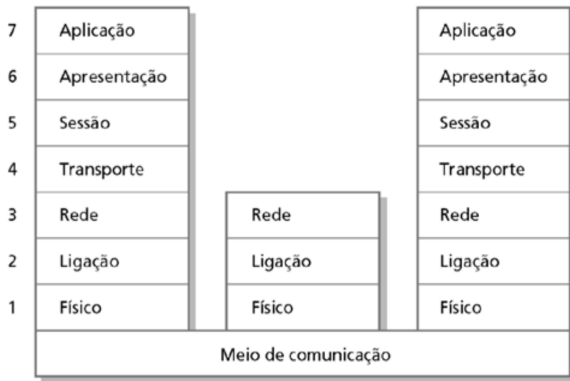
Arquitetura em Camadas

- Sistema organizado em camadas.
- Cada camada oferece serviços à superior e consome da inferior.



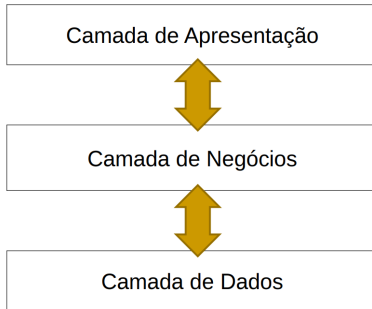
Exemplos de Arquitetura em Camadas

■ Modelo OSI (Redes)



Exemplos de Arquitetura em Camadas

- Aplicações web em 3 camadas:
 - Apresentação
 - Lógica de Negócios
 - Dados



Arquitetura em Camadas

- Vantagens:

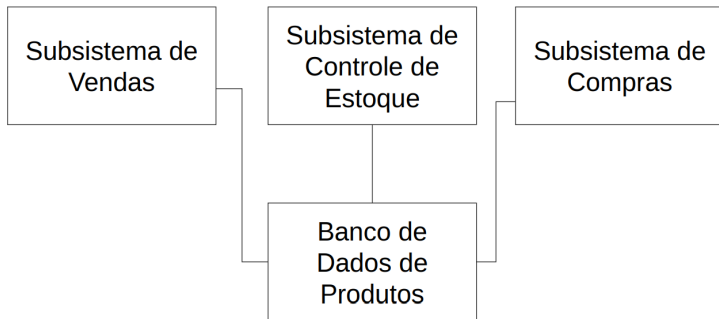
- Suporta desenvolvimento incremental.
- Facilita substituição de camadas com interfaces estáveis.
- Maior independência entre camadas.

- Desvantagens:

- Dificuldade para definir serviços das camadas inferiores.
- Muitas camadas podem comprometer o desempenho.

Arquitetura de Repositório (Blackboard)

- Subsistemas compartilham uma base de dados comum.
- Alguns escrevem, outros consomem dados.
- Adotado principalmente quando dados são compartilhados em grandes quantidades.



Arquitetura de Repositório (Blackboard)

■ Vantagens:

- Compartilhamento eficiente de dados.
- Facilidade de backup e integração.
- Possibilidade de proteger dados centralmente.

■ Desvantagens:

- Subsistemas precisam entender o formato dos dados.
- Dificuldade de evoluir grandes volumes de dados.
- Dificuldade para distribuir os dados – problemas de redundância e inconsistência.

Dutos e Filtros

- Dinâmica de fluxo de dados.
- Componentes:
 - Dutos: conduzem dados.
 - Filtros: transformam dados.
- Aplicável a processamento de dados sequencial ou concorrente.



Dutos e Filtros

- Vantagens:

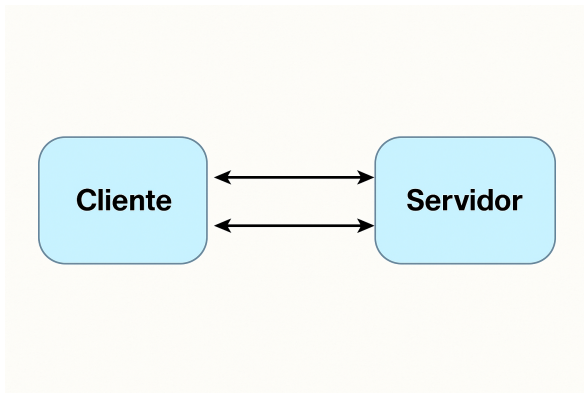
- Filtros são altamente reutilizáveis e modulares.
- Facilita a evolução incremental.
- Aderente a fluxos de trabalho.

- Desvantagens:

- Necessidade de padronização dos dados.
- Overhead de transformação e compatibilidade de formato.

Arquitetura Cliente-Servidor

- Dois tipos principais de componentes:
 - **Servidores:** oferecem serviços centralizados.
 - **Clientes:** requisitam e consomem os serviços.
- A comunicação é feita por meio de chamadas de serviço pela rede.



Arquitetura Cliente-Servidor

■ Vantagens:

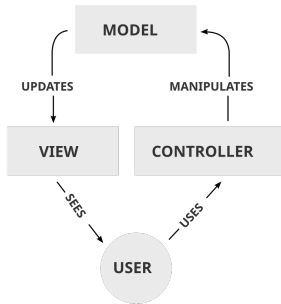
- Escalável: pode-se adicionar mais clientes ou servidores.
- Separação clara entre apresentação (cliente) e lógica/armazenamento (servidor).
- Suporte a múltiplas plataformas e dispositivos.

■ Desvantagens:

- O servidor pode se tornar um gargalo ou ponto único de falha.
- Exige gerência de conexões e segurança na comunicação.
- Alterações na lógica de negócio requerem mudanças no servidor.

Arquitetura MVC (Model-View-Controller)

- Separação de responsabilidades em três componentes:
 - **Modelo:** lógica de negócio e dados.
 - **Visão:** apresentação da interface ao usuário.
 - **Controlador:** coordena a interação entre modelo e visão.



Vantagens do MVC

- Vantagens:

- Promove separação de preocupações e facilita a manutenção.
- Permite múltiplas visualizações de um mesmo modelo.
- Facilita testes e reutilização de componentes.

- Desvantagens:

- Pode ser excessivamente complexo para sistemas pequenos.
- Comunicação entre componentes pode se tornar difícil de manter.
- Requer disciplina no projeto para manter a separação clara.

Revisão

- **Projeto Arquitetural:** primeira etapa do projeto, conecta requisitos e projeto detalhado.
- **Diagramas de Componentes:** representação da estrutura do sistema, facilitando comunicação, planejamento e desenvolvimento em paralelo.
- **Impactos da Arquitetura:** desempenho, distribuição, manutenção, escalabilidade, segurança e disponibilidade.
- **Padrões Arquiteturais:** soluções reutilizáveis para problemas recorrentes de estrutura de software.
- **Exemplos Estudados:** Arquitetura em Camadas, Repositório (Blackboard), Dutos e Filtros, Cliente-Servidor, MVC (Model-View-Controller).
- **Importância da Escolha Arquitetural:** impacta diretamente requisitos não funcionais e a evolução do sistema.